

Function Composition Talk

Logan Barnett-Hoy

<2022-05-11 Wed>

Contents

1	Function Composition	1
1.1	Bespoke: Average	1
1.2	Composed: Average	2
1.3	Review: Functions as Data part 1	3
1.4	Review: Functions as Data part 2	3
1.5	Call Sites in JavaScript	4
1.6	Writing our own Higher Order Function: Map	5
1.7	Writing our own Higher Order Function: changeFirstLetter	5
1.8	Simplifying Promises	6
1.9	Currying to Change Arity	7
1.10	Conclusions	8
1.11	Questions?	8

1 Function Composition

Function composition is the act of treating functions like small lego bricks that you tie together. Composing simple, well known functions to achieve complex behavior is like having an advanced vocabulary used to discuss a complex topic.

If you want to try some of this out, I recommend Ramda's REPL for experimenting with different JavaScript features such as this.

1.1 Bespoke: Average

Here's some code for writing an **average** function. It computes an average value from a list of numbers. For reference, an average is all of the numbers added together and divided by the number of values.

```

function average(list) {
  let total = 0
  for(let i = 0; i < list.length; ++i) {
    total = total + list[i]
  }
  const result = total / list.length
  return result
}

// Test it.
console.log(average([1, 2, 3]))
console.log(average([1, 1, 1, 1]))
console.log(average([0, 1, -1]))
console.log(average([2,2,2,2,2,1]))

```

1.2 Composed: Average

Using function composition we can create **average** a little differently.

```

// We need to make some basic operations composable.
function add(x, y) { return x + y }
function divide(x, y) { return x / y }

function sum(list) {
  return list.reduce(add, 0)
}

function average(list) {
  return divide(sum(list), list.length)
}

// Test it.
console.log(average([1, 2, 3]))
console.log(average([1, 1, 1, 1]))
console.log(average([0, 1, -1]))
console.log(average([2,2,2,2,2,1]))

```

The results are the same as our bespoke version.
 The value here is not that there is necessarily less code.

Each function can be thought of independently and solved as an independent problem. Can you tell me if **add** is solid? What about **divide**? **sum** gets a little more tricky, but even then there's not much to it. when we get to **average** we are only calling two functions and using two parameters.

When you get really comfortable with functions like **map**, **reduce**, and **filter**, almost any operation can be composed using some combination of those three. They become the elementary particles of programming.

1.3 Review: Functions as Data part 1

JavaScript allows functions to be assigned to variables.

We call this “functions as first-class citizens”.

Here's how this looks:

```
function frobnicate(x) {  
  console.log('I frobnicated ' + x.toString())  
}
```

```
frobnicate(1) // Just runs frobnicate and passes it 1.
```

```
const doAThing = frobnicate
```

```
doAThing(2) // Runs frobnicate and passes it 2.
```

This isn't particularly useful but it demonstrates that we can assign a function to a variable. In fact when we write **function frobnicate...** we are saying **frobnicate** is a variable too.

1.4 Review: Functions as Data part 2

Normally when we use something like **map** we pass an **anonymous** function - meaning the function has no name. But what if we pass it a function with a name?

```
// Here, the body of frobnicate is written out as an anonymous function.  
[1, 2, 3].map(function(x) {  
  console.log('I frobnicated ' + x.toString())  
})  
  
console.log('Switching gears!')
```

```
function frobnicate(x) {
  console.log('I frobnicated ' + x.toString())
}

// Here we pass frobnicate as a variable.
[4, 5, 6].map(frobnicate)
```

This is the “how” of functional composition. If we can assign a function to a variable, that means we can do anything with a function that we can do with a variable. One of the things you can do with a variable is you can pass it to another function. Computer Science dweebs call a function that accepts a function as a parameter a “higher order function”. It’s just a function that takes a function.

1.5 Call Sites in JavaScript

In order to invoke a function called `foo` we write `foo()`. The `()` after a symbol (name) is the indication that this is a function being called. This is how JavaScript knows this is a call site as opposed to just accessing the variable’s value.

```
function foo() {
  console.log('foo called')
}

foo() // Works.
foo( ) // Works.
foo ( ) // Works.
foo(
) // Works.

const x = 1
try {
  x() // Oh noes!
} catch (e) {
  console.log('Error: ' + e)
}
```

If you try this on something that isn’t a function, you’ll see `<variable> is not a function.`

1.6 Writing our own Higher Order Function: Map

Let's make our own simple higher order function. One of the utilities of higher order functions is they are inherently abstract, which kind of means it's useless. But the function passed to the higher order function allows the higher order function to specialize while remaining abstract, and specialization is how we achieve usefulness.

In this case we will implement own version of `map` on `Array`.

`map` returns a new Array based on the original Array. Every element of that new Array has been transformed by a transformational function.

```
function map(f, originalList) {
  const newList = []
  for(let i = 0; i < originalList.length; ++i) {
    newList[i] = f(originalList[i])
  }
  return newList
}

console.log(map(function(x) { return x + 1; }, [1, 2, 3]))
function uppercase(s) {
  return s.toUpperCase()
}
console.log(map(uppercase, ['a', 'b', 'c']))
```

Here `f` is the function being passed, which is just any function. So long as it obeys the arity (number of arguments) and returns something, it works.

1.7 Writing our own Higher Order Function: changeFirstLetter

```
function uppercase(s) {
  return s.toUpperCase()
}
function lowercase(s) {
  return s.toLowerCase()
}

// Change the first letter of ever word, using f to do the transformation.
function changeFirstLetter(s, f) {
  const words = s.split(' ') // Split words.
```

```

    const newWords = words.map(function(w) {
      return f(w[0]) + w.slice(1)
    })
    return newWords.join(' ') // Put the words back into a sentence.
  }

  console.log(changeFirstLetter('i am too lazy to capitalize.', uppercase))
  console.log(changeFirstLetter('CAPSLOCK IS CRUSE CONTROL FOR AWESOME', lowercase))
  // Change it up - increment the character by 1.
  console.log(changeFirstLetter('I am a proper sentence actually.', function(c) {
    return String.fromCharCode(c.charCodeAt(0) + 1)
  })))

```

The beauty of this is `changeFirstLetter` doesn't need to know about what kind of changes could be made to the first letter. That has been delegated to the function argument `f`.

1.8 Simplifying Promises

Promises are tricky topics in JavaScript that many engineers struggle with. Since promises operate on functions they are provided, we can use composition to simplify parts of the promise. In some ways promise chains make function composition easier to understand.

```

// Fake, to simplify example.
function readFile(path) {
  return JSON.stringify({
    user: 'Me',
    accounts: [
      { totalMoney: 0 },
      { totalMoney: 10000 },
      { totalMoney: 2 },
    ],
  })
}

function add(x, y) { return x + y }

function computeAccounts(payload) {
  return payload
}

```

```

    .accounts
    .map(x => x.totalMoney)
    .reduce(add, 0)
  }

// Read an account file from a path and compute its total amount.
function accountTotal(path) {
  return Promise
    .resolve(path)
    // readFile could return a Promise instead of a concrete value, and this
    // could would remain unchanged.
    .then(readFile)
    .then(JSON.parse)
    .then(computeAccounts)
}

accountTotal('foo.json').then(amount => console.log(amount))

```

1.9 Currying to Change Arity

When doing function composition, you can use a technique called “currying” to provide different function arity (argument count). A function is said to be “curried” if it takes one argument and returns another function.

```

function add(x) {
  // This function just immediately returns a function.
  return function(y) {
    // The x variable is pulled in from the scope.
    return x + y
  }
}

console.log(add(1)(2))
console.log(add(5)(5))

// Then make a named function with it.
const addOne = add(1)
console.log(addOne(0))
console.log(addOne(1))

```

```
// Normally "add" has two arguments and therefore is not suitable for map. But
// with a curried add we can use it with map.
console.log([1, 2, 3].map(addOne))
// Same as above, but without the named version.
console.log([1, 2, 3].map(add(1)))
```

1.10 Conclusions

Function composition is a topic with a great deal of depth to it, but in an ecosystem like JavaScript you can break down almost any problem into tiny, reusable pieces. It takes some practice and starts getting easier with time as you build up a stronger vocabulary of functions for yourself.

1.11 Questions?